

Practice 17 详解答案

A. 单项选择题

1. **B**: 工厂模式将对象创建集中到专门位置。
2. **B**: 客户端把创建对象的工作委托给工厂。
3. **B**: 到处写 `new TextNote` 会让调用代码依赖具体类。
4. **C**: `NoteFactory` 负责决定并创建具体笔记类型。
5. **C**: 工厂通常返回基类 `Note*`，以便统一处理不同派生对象。
6. **A**: 工厂方法的职责是创建并返回对象。
7. **B**: 类只通过必要接口协作，不依赖彼此实现细节。
8. **A**: 创建逻辑集中后，更容易替换或添加具体实现。
9. **A**: 对象创建与对象使用分离。

B. 填空题

1. 创建
2. 耦合
3. 基
4. 扩展
5. 直接

C. 判断题

1. **对**: 工厂负责创建策略；对象创建后如何工作仍由对象自身定义。
2. **错**: 工厂内部通常正是通过构造函数创建对象。
3. **对**: 客户端通常只使用基类接口，不必知道具体派生类。
4. **对**: 例如工厂返回 `TextNote*` 或 `TodoNote*`，但返回类型可统一为 `Note*`。
5. **对**: 创建规则集中后，修改地点更少。
6. **错**: 项目可以有零个、一个或多个工厂，取决于实际需求。
7. **对**: 例如工厂中写 `return new Dog;`。
8. **对**: 工厂创建基类指针，多态决定后续虚函数调用。
9. **错**: 对象创建简单且稳定时，直接构造更清晰；不应机械使用工厂。
10. **对**: 新增类型主要集中修改工厂，而不是所有使用对象的地方。

D. 代码阅读

1. NoteFactory 的职责

```
class NoteFactory
{
public:
    Note* createNote();
};
```

它负责创建并返回 `Note` 对象（通常是某个派生类对象）。对象使用者只调用 `createNote()`，不需要知道内部选择了哪种具体笔记。

2. 展示的设计原则

```
Note* note = factory.createNote();
```

这体现了**工厂模式、低耦合和依赖抽象**。调用代码依赖 `Note*`，而不是直接依赖 `TextNote`。

3. 最可能返回的对象

```
Note* note = factory.create("todo");
```

最可能返回 `TodoNote` 对象的地址，即它的静态类型是 `Note*`，实际对象类型是 `TodoNote`。

4. 为什么在 `main()` 中判断不灵活

`main()` 同时承担菜单流程和创建具体对象的职责。新增 `ReminderNote` 时必须修改 `main()`；若其他地方也有相同判断，也都要修改。把判断放进 `NoteFactory::create()` 后，创建规则只有一处。

5. 展示的关系

```
Note* note = factory.create(type);
note->display();
```

展示了**工厂模式与多态的配合**：工厂返回基类指针，`display()` 作为虚函数会执行实际派生类自己的实现。

E. 简答题

1. 为什么大型系统常用工厂模式

大型系统对象类型多、创建规则可能复杂。工厂把参数校验、类型选择和构造过程集中起来，避免 `new` 分散在菜单、业务逻辑和其他模块中。客户端只面向基类接口工作，创建方式改变时受影响的代码更少。

2. 工厂如何支持开闭原则

客户端依赖 `Note` 和 `NoteFactory::create()`，而不是依赖 `TextNote`、`TodoNote` 等具体类型。增加一种派生类时，通常只需新增类并扩展工厂的类型选择分支；已有的 `Note*` 容器和 `display()` 循环不需要修改。因此系统更容易对新功能扩展。

3. 工厂与多态如何配合

工厂根据输入创建具体派生对象，但用基类指针返回：

```
Note* note = factory.create("todo");
note->display();
```

创建阶段由工厂决定实际类型；使用阶段只调用 `Note` 的虚函数，由多态执行 `TodoNote::display()` 等正确版本。

F. 编程题

1. 第一个动物工厂

```
#include <iostream>
#include <string>
using namespace std;

class Animal
{
public:
    virtual void speak() const = 0;
    virtual ~Animal() = default;
};

class Dog : public Animal
{
public:
    void speak() const override
    {
        cout << "Woof\n";
    }
};

class Cat : public Animal
{
public:
    void speak() const override
    {
        cout << "Meow\n";
    }
};

class AnimalFactory
{
public:
    Animal* createAnimal(const string& type)
    {
        if (type == "dog") {
            return new Dog;
        }
        if (type == "cat") {
            return new Cat;
        }
        return nullptr;
    }
};
```

```

    }
};

int main()
{
    AnimalFactory factory;
    string type;

    cout << "Enter dog or cat: ";
    cin >> type;

    Animal* animal = factory.createAnimal(type);
    if (animal == nullptr) {
        cout << "Unknown animal.\n";
        return 0;
    }

    animal->speak();
    delete animal;
}

```

`animal` 的类型是 `Animal*`，但它可指向 `Dog` 或 `Cat`。最后的 `delete` 释放工厂用 `new` 创建的对象。

2. Shape 工厂

```
#include <iostream>
#include <string>
using namespace std;

class Shape
{
public:
    virtual void draw() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape
{
public:
    void draw() const override
    {
        cout << "Drawing a circle.\n";
    }
};

class Rectangle : public Shape
{
public:
    void draw() const override
    {
        cout << "Drawing a rectangle.\n";
    }
};

class ShapeFactory
{
public:
    Shape* createShape(const string& type)
    {
        if (type == "circle") {
            return new Circle;
        }
        if (type == "rectangle") {
            return new Rectangle;
        }
        return nullptr;
    }
};
```

```
int main()
{
    ShapeFactory factory;
    Shape* shape = factory.createShape("circle");

    if (shape != nullptr) {
        shape->draw();
        delete shape;
    }
}
```

3. 重构对象创建

```
class NoteFactory
{
public:
    Note* create(const string& type)
    {
        if (type == "text") {
            return new TextNote;
        }
        if (type == "todo") {
            return new TodoNote;
        }
        return nullptr;
    }
};
```

使用处变为：

```
NoteFactory factory;
Note* note = factory.create(type);
```

这样 main() 不再直接知道 TextNote 和 TodoNote 的构造细节；类型判断集中在工厂，创建规则不会散落和重复。

4. 扩展工厂

```
class ReminderNote : public Note
{
public:
    void display() const override
    {
        cout << "Reminder note\n";
    }
};

class NoteFactory
{
public:
    Note* create(const string& type)
    {
        if (type == "text") {
            return new TextNote;
        }
        if (type == "todo") {
            return new TodoNote;
        }
        if (type == "reminder") {
            return new ReminderNote;
        }
        return nullptr;
    }
};
```

客户端仍使用原来的代码：

```
Note* note = factory.create(type);
note->display();
```

无需让 main() 新增 if 判断。

5. CLI Notes 工厂（挑战）

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Note
{
public:
    virtual void display() const = 0;
    virtual ~Note() = default;
};

class TextNote : public Note
{
public:
    void display() const override
    {
        cout << "[Text note]\n";
    }
};

class TodoNote : public Note
{
public:
    void display() const override
    {
        cout << "[Todo note]\n";
    }
};

class ReminderNote : public Note
{
public:
    void display() const override
    {
        cout << "[Reminder note]\n";
    }
};

class NoteFactory
{
public:
    Note* create(const string& type)
    {
```

```

        if (type == "text") {
            return new TextNote;
        }
        if (type == "todo") {
            return new TodoNote;
        }
        if (type == "reminder") {
            return new ReminderNote;
        }
        return nullptr;
    }
};

```

```

class NoteManager
{
private:
    vector<Note*> notes;

public:
    ~NoteManager()
    {
        for (int i = 0; i < notes.size(); i++) {
            delete notes[i];
        }
    }

    void addNote(Note* note)
    {
        if (note != nullptr) {
            notes.push_back(note);
        }
    }

    void displayAll() const
    {
        for (int i = 0; i < notes.size(); i++) {
            notes[i]->display();
        }
    }
};

```

```

int main()
{
    NoteFactory factory;
    NoteManager manager;
    int choice = 0;

```

```

while (choice != 4) {
    cout << "1. Text\n2. Todo\n3. Reminder\n4. Display and exit\n";
    cout << "Choose: ";
    cin >> choice;

    string type;
    if (choice == 1) {
        type = "text";
    } else if (choice == 2) {
        type = "todo";
    } else if (choice == 3) {
        type = "reminder";
    } else if (choice == 4) {
        break;
    } else {
        cout << "Invalid choice.\n";
        continue;
    }

    manager.addNote(factory.create(type));
}

manager.displayAll();
}

```

- NoteFactory 创建实际的派生对象。
- NoteManager 只保存和显示 Note*，不关心实际类型。
- NoteManager 的析构函数统一 delete 所有保存的对象，防止内存泄漏。

为什么 unique_ptr<Note> 更安全： unique_ptr 离开作用域时会自动 delete 对象，就不需要自己写析构函数和循环删除，也更不容易忘记释放内存。

G. 设计与思考挑战

设计 B 更好，因为创建具体对象的判断集中在 NoteFactory。客户端只依赖 Note* 和 create(type)，不依赖各个派生类的名字。新增笔记类型时，主要修改工厂并新增派生类；菜单、管理器和显示循环仍使用共同的 Note 接口。这减少重复、降低耦合并提高可维护性。